

5.2.1 Memoizing streams and avoiding recomputation

We typically want to cache the values of a `Cons` node, once they are forced. If we use the `Cons` data constructor directly, for instance, this code will actually compute `expensive(x)` twice:

```
val x = Cons(() => expensive(x), tl)
val h1 = x.headOption
val h2 = x.headOption
```

We typically avoid this problem by defining *smart* constructors, which is what we call a function for constructing a data type that ensures some additional invariant or provides a slightly different signature than the “real” constructors used for pattern matching. By convention, smart constructors typically lowercase the first letter of the corresponding data constructor. Here, our `cons` smart constructor takes care of memoizing the by-name arguments for the head and tail of the `Cons`. This is a common trick, and it ensures that our `thunk` will only do its work once, when forced for the first time. Subsequent forces will return the cached lazy `val`:

```
def cons[A](hd: => A, tl: => Stream[A]): Stream[A] = {
  lazy val head = hd
  lazy val tail = tl
  Cons(() => head, () => tail)
}
```

The empty smart constructor just returns `Empty`, but annotates `Empty` as a `Stream[A]`, which is better for type inference in some cases.⁴ We can see how both smart constructors are used in the `Stream.apply` function:

```
def apply[A](as: A*): Stream[A] =
  if (as.isEmpty) empty
  else cons(as.head, apply(as.tail: _*))
```

Again, Scala takes care of wrapping the arguments to `cons` in `thunks`, so the `as.head` and `apply(as.tail: _*)` expressions won’t be evaluated until we force the `Stream`.

5.2.2 Helper functions for inspecting streams

Before continuing, let’s write a few helper functions to make inspecting streams easier.



EXERCISE 5.1

Write a function to convert a `Stream` to a `List`, which will force its evaluation and let you look at it in the REPL. You can convert to the regular `List` type in the standard library. You can place this and other functions that operate on a `Stream` inside the `Stream` trait.

```
def toList: List[A]
```

⁴ Recall that Scala uses subtyping to represent data constructors, but we almost always want to infer `Stream` as the type, not `Cons` or `Empty`. Making smart constructors that return the base type is a common trick.